

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Weightage:	30% of CIA
Student Name:	G Hari Prasad	Reg. No.:	192525273
Section / Batch:	B	Date:	24-08-2026

Q1. Subset Sum Problem with Exact K Elements

Design a pseudocode using Backtracking to determine whether there exists a subset of exactly k elements whose sum equals a target value. Clearly identify inputs (set of integers, target sum, value k) and output (true/false or valid subset). Develop an algorithm that explores inclusion and exclusion of elements while tracking subset size and current sum. Use loops, recursive calls, and conditional checks effectively during subset construction. Apply pruning when the partial sum exceeds the target or the subset size exceeds k . Ensure the algorithm terminates correctly and analyze its complexity.

1. Problem Understanding

Inputs: a set/array S of n integers; a target sum value "target"; an integer k specifying the required subset size.

Output: TRUE together with a witnessing subset of exactly k elements whose sum equals target, if such a subset exists; FALSE otherwise.

Constraints: exactly k elements must be selected (neither more nor fewer); each element of S can be used at most once (0/1 selection, no repetition); the sum of the selected elements must equal target exactly.

This is a decision-style combinatorial search problem: for every element we make a binary choice – include it in the subset or exclude it – while two running quantities, the partial sum and the partial subset size, must both stay within their respective bounds throughout the construction.

2. State-Space Representation

Each element of S is either included or excluded, so the raw search space is the binary decision tree of depth n with 2^n leaves (subsets). A state is represented by the tuple (start, currentSum, count), where *start* is the index of the next candidate element to consider, *currentSum* is the sum of elements chosen so far, and *count* is the number of elements chosen so far. The root state is (0, 0, 0); a leaf is reached either when count reaches k (subset complete) or when all n elements have been considered.

3. Backtracking Pseudocode

```
ALGORITHM SubsetSumExactK( $S$ , target,  $k$ )
  Input :  $S[0..n-1]$  - set/array of  $n$  integers
         target      - required sum (integer)
          $k$           - required subset size (integer)
  Output: TRUE and the witnessing subset, if one exists
         FALSE, otherwise

  chosen <- empty list           // elements picked so far
```

```

PROCEDURE Backtrack(start, currentSum, count):
    // ---- Success condition ----
    IF count == k AND currentSum == target:
        PRINT "TRUE", chosen
        RETURN TRUE

    // ---- Pruning conditions ----
    IF count >= k OR currentSum >= target:
        RETURN FALSE // size or sum bound already violated

    IF (n - start) < (k - count):
        RETURN FALSE // not enough elements remain to reach size k

    // ---- Explore: include / exclude each remaining element ----
    FOR i <- start TO n-1:
        chosen.APPEND(S[i]) // include S[i]
        IF Backtrack(i + 1, currentSum + S[i], count + 1) == TRUE:
            RETURN TRUE
        chosen.REMOVE_LAST() // exclude S[i] (undo)

    RETURN FALSE

RETURN Backtrack(0, 0, 0)

```

4. Explanation of Control Structures Used

Construct	Role
Recursive call	Backtrack(i+1, currentSum+S[i], count+1) explores the "include S[i]" branch; control returns to the FOR loop to try the next index, implicitly exploring the "exclude" branch for S[i].
FOR loop	Iterates i from start to n-1, generating every remaining candidate to extend the current partial subset - this is what produces all combinations without repetition (i+1 ensures no element is reused and avoids duplicate subsets).
IF conditional checks	Used for the success condition (count==k and currentSum==target) and for both pruning conditions, before any recursive descent is attempted.
APPEND / REMOVE_LAST	Push the candidate onto the partial subset before recursing, and pop it off after recursion returns - the classic 'choose, explore, un-choose' backtracking pattern.

5. Pruning Conditions

Pruning rule	Condition and reason
Size bound	$\text{count} \geq k \rightarrow$ stop extending this branch; adding more elements would exceed the required subset size k .
Sum bound	$\text{currentSum} \geq \text{target} \rightarrow$ stop extending this branch; since all inputs are treated as positive integers here, any further inclusion can only increase the sum past target.
Feasibility bound	$(n - \text{start}) < (k - \text{count}) \rightarrow$ stop; the number of elements left to examine is smaller than the number still needed to reach size k , so this branch cannot possibly succeed.

Together, these three checks cut off entire subtrees as soon as a partial solution can no longer lead to a valid answer, which is far more efficient than generating all 2^n subsets and testing each one afterwards.

6. Recursion Trace (Dry Run)

Input: $S = [3, 7, 1, 9, 4, 6, 2]$, $\text{target} = 12$, $k = 3$

```
Backtrack(start=0, sum=0, count=0)
|-- include 3 -> Backtrack(1, 3, 1)
|   |-- include 7 -> Backtrack(2, 10, 2)
|   |   |-- include 1 -> sum=11, count=3 -> FAIL (sum!=target) -> undo
|   |   |-- include 9 -> sum>=target(19) -> PRUNE -> undo
|   |   |-- include 4 -> sum>=target(14) -> PRUNE -> undo
|   |   |-- include 6 -> sum>=target(16) -> PRUNE -> undo
|   |   |-- include 2 -> sum=12, count=3 -> SUCCESS -> {3, 7, 2}
|   |
|   |
|--
result: TRUE, subset = [3, 7, 2]
```

The trace shows the search reaching a valid answer after only a handful of recursive calls because the sum-bound pruning rule immediately discards the branches that include 9, 4, or 6 once the partial sum would exceed target – the search never has to explore those subtrees at all.

7. Correctness Justification

Completeness: without pruning, the FOR loop combined with recursion enumerates every combination of elements (every subset), so the search would eventually try every size- k subset and correctly report TRUE if any of them sums to target. **Soundness of pruning:** each pruning condition only discards branches that provably cannot yield a valid solution – exceeding k or target can never be corrected by adding more (non-negative) elements, and having fewer remaining elements than still-needed slots makes reaching size k impossible – so no valid solution is ever lost. **Termination:** every recursive call strictly increases start , and start is bounded above by n , so the recursion depth is bounded and the algorithm always terminates, correctly returning TRUE with a witnessing subset or FALSE if none exists.

8. Complexity Analysis

Worst case – $O(C(n,k) \cdot k)$: in the worst case (e.g. no early sum-based pruning triggers, such as when all elements are small and positive but the target is never hit until the very last combination) the algorithm explores close to $C(n,k) = n!/(k!(n-k)!)$ size- k combinations, each built in $O(k)$ time, giving worst-case time complexity $O(C(n,k) \cdot k)$, which for $k = \Theta(n)$ is $O(2^n)$ in the fully exponential worst case (Big-O, upper bound).

Best case – $\Omega(k)$: if the very first k elements considered already sum to target, the algorithm succeeds after exactly k recursive calls, giving best-case time $\Omega(k)$ (Big-Omega, lower bound).

Average / typical case with pruning – $\Theta(\text{pruned search space})$: the sum-bound and feasibility-bound pruning conditions eliminate large portions of the search tree in practice, so the number of nodes actually visited is data-dependent; it is tightly bounded, $\Theta(\text{nodes actually explored})$, which is generally far smaller than the raw $C(n,k)$ bound but cannot be improved to polynomial time in the worst case for arbitrary inputs.

Space complexity: $O(k)$ for the recursion stack and the "chosen" list, excluding the $O(n)$ input storage.

9. Complexity Class Discussion

The decision version of this problem – "does there exist a size- k subset of S summing to target?" – belongs to **NP**: given a candidate subset (certificate) of size k , its size and sum can both be checked in $O(k)$ time, which is polynomial, so a "yes" answer is verifiable in polynomial time. It is also **NP-hard**, since the classical Subset Sum problem (without the exact- k restriction) is a well-known NP-Complete problem and reduces to this exact- k variant in polynomial time (any Subset Sum instance can be solved by trying this exact- k algorithm for every value of k from 0 to n); hence the exact- k version is at least as hard as Subset Sum. Combining both results, the problem is **NP-Complete**. It is worth noting that Subset Sum is only **weakly NP-Complete**: a pseudo-polynomial dynamic-programming solution exists with time complexity $O(n \cdot k \cdot \text{target})$, which is efficient when target and k are small, even though no solution polynomial purely in the input size (bit-length) is known. The backtracking approach above is the natural exact/exhaustive strategy consistent with this NP-Complete classification, accelerated in practice by the pruning conditions described above.

10. Python Implementation (Supplementary Verification)

The pseudocode above was additionally implemented and executed to verify its correctness on a sample instance:

```
"""
Subset Sum with Exactly K Elements - Backtracking with Pruning
"""

def read_input():
    nums = [3, 7, 1, 9, 4, 6, 2]
    target = 12
    k = 3
    return nums, target, k

def subset_sum_exact_k(nums, target, k):
    n = len(nums)
    chosen = []
    result = {"found": False, "subset": None}

    def backtrack(start, current_sum, count):
        if result["found"]:
            return
        # success condition
        if count == k and current_sum == target:
            result["found"] = True
            result["subset"] = chosen.copy()
            return
        # PRUNING: subset size exceeded, or sum already exceeded target
        if count >= k or current_sum >= target:
            return
        # PRUNING: not enough elements left to reach k
        if (n - start) < (k - count):
            return

        for i in range(start, n):
            chosen.append(nums[i])
            backtrack(i + 1, current_sum + nums[i], count + 1)
            chosen.pop()
            if result["found"]:
                return

    backtrack(0, 0, 0)
    return result["found"], result["subset"]

if __name__ == "__main__":
    nums, target, k = read_input()
    print("Input set      :", nums)
    print("Target sum     :", target)
    print("k (size)        :", k)
    print("-" * 50)

    found, subset = subset_sum_exact_k(nums, target, k)
    if found:
        print(f"Result: TRUE - subset found -> {subset} "
              f"(sum = {sum(subset)}, size = {len(subset)})")
    else:
        print("Result: FALSE - no subset of size k sums to target")
```

11. Program Output

```
Input set      : [3, 7, 1, 9, 4, 6, 2]
Target sum     : 12
k (size)       : 3
-----
Result: TRUE - subset found -> [3, 7, 2] (sum = 12, size = 3)
```

Interpretation: For $S = [3, 7, 1, 9, 4, 6, 2]$, $\text{target} = 12$, $k = 3$, the algorithm correctly identifies the subset $\{3, 7, 2\}$ ($\text{sum} = 12$, $\text{size} = 3$), confirming both the pseudocode logic and the pruning conditions behave as

intended.

12. Conclusion

A backtracking pseudocode for the exact-k Subset Sum problem was developed with clearly identified inputs, outputs, and constraints, a formal state-space representation, well-structured recursive control flow, and two effective pruning conditions on subset size and partial sum. The algorithm was shown to be correct (complete, sound, and terminating), its complexity was analyzed using Big-O, Big-Omega and Big-Theta notation, and the underlying decision problem was classified as NP-Complete, consistent with why an exact backtracking search (rather than a known polynomial-time algorithm) is the appropriate exact solution strategy.

I **G Hari Prasad (Reg. No. 192525273)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the Student: **G Hari Prasad**

Date: **24-08-2026**